

Aula 10: Mais coleções em Python

Manipulando tuplas, conjuntos e dicionários

Maurício da Silva Pires
mspires@id.uff.br

TCC00326 - Programação de computadores
Niterói, Rio de Janeiro

Revisitando as aulas anteriores

- ⌋ Nas aulas anteriores, trabalhamos com o objetos iteráveis no Python
 - * Discutimos listas lineares e *strings*
 - * Falamos sobre suas semelhanças e diferenças
 - * Diferenciamos funções recursivas de iterativas
- ⌋ Python oferece outros tipos de objetos iteráveis que possuem características interessantes
- ⌋ Nesta aula, vamos explorar outros 3 tipos de iteráveis: **tuplas**, **conjuntos** e **dicionários**

- ﴿ **Iteráveis** é o nome dado a objetos capazes de retornar seus elementos um por vez, permitindo assim serem processados por uma repetição **for**
 - * Além das *strings* e dos intervalos de inteiros (*range*), o Python possui uma série de coleções nativas que são iteráveis
 - * Listas lineares (*list*), conjuntos (*set*), tuplas (*tuple*) e dicionários (*dict*)
- ﴿ Tais coleções não precisam, necessariamente, ser homogêneas, ou seja, seus elementos não precisam ser todos de um mesmo tipo, como *strings* ou números reais
- ﴿ Podemos definir coleções de coleções também

Tuplas

- ☾ As tuplas são sequências imutáveis que podem armazenar um número qualquer de cópias de um mesmo elemento
 - * Definimos uma tupla com parênteses e os elementos listados separados por vírgulas
 - * Todas as operações que “alteram” a estrutura da tupla cria uma nova tupla

```
a = () # tupla vazia
```

```
b = (1,) # tupla unitária  
# é obrigatória a vírgula depois do elemento
```

```
# tupla com 4 elementos  
c = (1, 3, 14, "aba")
```

Tuplas

- ⌋ Os elementos de uma tupla são acessados a partir de seus índices
 - * Possuem a mesma dinâmica de acesso de uma lista linear
 - * Os índices das tuplas de n elementos começam em 0 e terminam em $n - 1$
 - * O acesso da direita para esquerda é feito com números negativos a partir de -1 (posição mais à direita)

```
tupla = ("abacate", "melão", "pêssego")
```

```
print(tupla[0])      # imprime abacate
print(tupla[-1])     # imprime pêssego
print(tupla[2])      # imprime pêssego
```

Criação de uma tupla

﴿ Podemos criar uma tupla de duas formas

- * Atribuindo a uma variável os parênteses, listando os elementos ou não
- * Usando a função construtora **tuple()**

﴿ A função **tuple()** pode receber no máximo um argumento. Isto significa que só podemos criar tuplas vazias ou converter outras “coleções” em tuplas

```
tupla1 = []  
tupla2 = tuple()  
tupla1 == tupla2           # True  
tupla3 = tuple("lua")      # ("l", "u", "a")
```

Tamanho de uma tupla

☾ Para saber quantos elementos existem em uma tupla usamos a função **len**

✱ Para saber se a tupla está vazia, verificamos se o tamanho da tupla é zero

```
tupla1 = (2,-9,0,14,"doce")  
print(len(tupla1))           # 5
```

```
tupla2 = (0,1,2,3,4,5)  
print(len(tupla2))           # 6
```

```
tupla3 = tuple("abacate")  
print(len(tupla3))           # 7
```

Busca de dados

- ⌋ A busca por um dado em uma tupla é um teste de pertinência com o operador **in**
 - * O operador retorna **True** se o elemento existe na tupla e **False** caso contrário
 - * A função *index(element)* retorna o índice do elemento na tupla
 - * Se o elemento em questão não existir na tupla, a função gera um erro

```
tupla = (2,3,4,5,6)
print(5 in tupla)           # True
print(10 in tupla)          # False
print(tupla.index(5))       # 3
print(tupla.index(10))      # Gera um erro
```

⌋ As tuplas são imutáveis

- * Impossível adicionar ou remover elementos na tupla
- * Alterações podem ser feitas mediante conversões de tuplas para listas e depois de volta para tuplas
- * A depender da estrutura de dados, é mais interessante manter todo o processamento em cima de listas e retornar para o tipo de tupla apenas na hora de retornar ou apresentar os dados

⌋ O Python as armazena de forma contínua como uma sequência de referências (ponteiros) para os objetos reais, e não os objetos em si

- * Por serem imutáveis, o Python aplica otimizações de desempenho, tornando-as mais rápidas e econômicas no uso de memória do que as listas

Vantagens do uso de tuplas

- Internamente, uma tupla é um array estático e compacto de ponteiros. Cada posição da tupla guarda apenas o endereço de memória do objeto real que ela contém
- Ao contrário das listas, as tuplas não exigem espaço de reserva extra para acomodar futuras adições de elementos. Isso resulta em uma estrutura mais leve e eficiente para o sistema
 - ✱ Redução do custo de processamento (sobrecarga, *overhead*)
- Para tuplas pequenas com valores imutáveis comuns (como inteiros pequenos ou *strings* curtas), o Python pode até reutilizar os mesmos blocos de memória ou referências em vez de criar novos

Outras operações

- ⌋ As operações de $+$ e $*$ também estão disponíveis para tuplas e se comportam de forma similar ao que ocorre nas listas
- ⌋ O operador **not in** também funciona com tuplas, ou seja, retorna **True** se o elemento não está na tupla e **False** se o elemento estiver na tupla
- ⌋ O método *count(element)* é chamado a partir da tupla e retorna o número de cópias do elemento na tupla
- ⌋ As tuplas, assim como as listas e *strings*, também estão sujeitas à aplicação de fatiamento
 - * Fatiar uma tupla gera uma nova tupla

Aplicação de tuplas

- ⌋ As tuplas são perfeitas para agrupar informações relacionadas a uma única entidade, como os dados de um cliente ou as coordenadas geográficas de um mapa
- ⌋ Quando uma função precisa retornar mais de um resultado, ela empacota esses valores em uma tupla
 - * Os elementos de retorno são separados por vírgulas
- ⌋ Se você possui um conjunto de valores fixos que nunca devem mudar no seu programa, as tuplas evitam erros de lógica acidentais
- ⌋ Diferentemente das listas (que são mutáveis), as tuplas são consideradas objetos *hashable* no Python, podendo assim serem usadas como chaves de dicionários

Conjuntos

- ⌋ Conjuntos são coleções não-ordenadas de elementos distintos
 - * Similar ao que fazemos na Matemática, as chaves delimitam o escopo de um conjunto e, entre elas, estão seus elementos listados
 - * Por convenção, um conjunto vazio é criado sempre a partir do construtor **set()**

```
a = set() # conjunto vazio
```

```
b = {1} # conjunto unitário
```

```
# conjunto com 4 elementos
```

```
c = {1, 3, 14, "aba"}
```

Conjuntos

﴿ Visto que os elementos de um conjunto não são indexados, as únicas formas de acesso são

- ✱ **Teste de pertinência:** verifica se um elemento pertence (**in**) ou não pertence (**not in**) ao conjunto
- ✱ **Iteração:** acessamos diretamente cada um dos elementos do conjunto a partir de um laço **for**

```
conj = {"abacate", "melão", "pêssego"}

print("melão" in conj)           # True
print("abóbora" not in conj)    # True
for elmt in conj:
    print(elmt)                  # Imprime um
                                # elemento por
                                # vez
```

Criação de um conjunto

☾ Podemos criar um conjunto de 4 formas

- * Um conjunto vazio a partir do construtor **set()**
- * Atribuindo a uma variável as chaves e listando os elementos
- * Convertendo algum iterável em um conjunto usando **set()**
- * Usando *set comprehension*

```
C1 = set()                                # conjunto vazio
palavra = "casados"
C2 = set(palavra)                         # {"c","a","s","d","o"}
C3 = {1,2,3,4,5,2,1}                     # {1,2,3,4,5}
```

Set comprehension

- ⌋ Podemos definir um conjunto a partir de uma outra coleção usando o seguinte modelo

`{valor for variável in coleção if condição}`

- * **valor** é a expressão que define os dados do conjunto
- * **variável** é a variável usada no laço **for**
- * **coleção** é a coleção usada como base
- * **condição** é uma condição que os valores de **variável** devem atender (parte opcional)

```
lista = [13,14,16,49,36,54,81]
C = {x for x in lista if int(x**0.5) == x**0.5}
print(C)           # {16,49,36,81}
```

Inserção de dados

- ⌋ A inserção de um dado em conjuntos Python pode ocorrer de duas maneiras diferentes
 - * **Inserir um único elemento:** a partir do conjunto, chamamos o método *add(element)* para incluir *element* no conjunto
 - * Se o elemento inserido já existir no conjunto, apenas uma cópia dele permanecerá disponível. Quaisquer duplicatas são descartadas
 - * **Inserir vários elementos:** a partir do conjunto, chamamos o método *update(iterable)*, que recebe um objeto iterável que será a fonte dos elem

```
C = {1, 2, 3, 4}
C.add(5)           # {1, 2, 3, 4, 5}
C.add(2)           # {1, 2, 3, 4, 5}
C.add([5, 6, 2, 3]) # {1, 2, 3, 4, 5, 6}
```

Remoção de dados

- Os dados de um conjunto podem ser removidos de 4 maneiras diferentes
 - O método *remove(element)* remove o elemento e gera um erro se ele não existir no conjunto
 - O método *discard(element)* remove o elemento e **não** gera erro caso o elemento não exista no conjunto
 - O método *pop()* remove e retorna um elemento qualquer do conjunto. Gera um erro se o conjunto estiver vazio
 - O método *clear()* é utilizado para remover todos os elementos do conjunto, deixando-o vazio

```
C = {1,2,3,4,5,6}
C.pop()           # 4
C.remove(10)      # Gera erro
C.discard(10)     # {1,2,3,5,6}
```

Operações nativas

⌋ Conjuntos, enquanto estruturas de dados, possuem as mesmas operações que na matemática

✱ As operações mais clássicas sobre conjuntos são a união ($\cup$), a interseção ($\cap$) e a diferença de conjuntos ($-$)

✱ O operador $|$ computa $A \cup B = \{x \mid x \in A \text{ ou } x \in B\}$

✱ O operador $\&$ computa $A \cap B = \{x \mid x \in A \text{ e } x \in B\}$

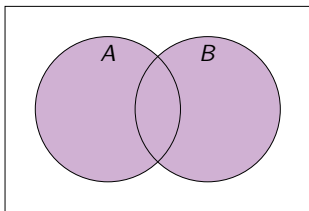
✱ O operador $-$ computa $A - B = \{x \in A \mid x \notin B\}$

⌋ Uma operação um pouco mais avançada disponível é a diferença simétrica (Δ) a partir do operador $\wedge$

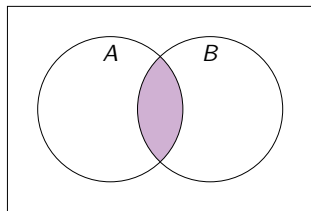
$$A \Delta B = \{x \mid x \in A \cup B \text{ e } x \notin A \cap B\}$$

Diagramas de Venn

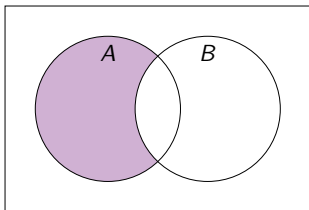
$$A \cup B$$



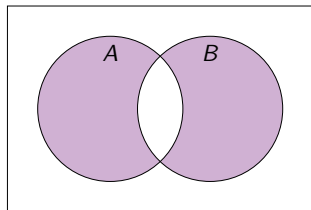
$$A \cap B$$



$$A - B$$



$$A \Delta B$$



Exemplo de operações nativas

A = {1, 2, 5, 6, 7}

B = {2, 3, 4, 5, 1, 7}

C1 = A | B # {1, 2, 3, 4, 5, 6, 7}

C2 = A & B # {1, 2, 5, 7}

C3 = A ^ B # {3, 4, 6}

Operações comutativas

C4 = A - B # {6}

C5 = B - A # {3, 4}

Operação não-comutativa

Outras operações

- ⌈ Determinamos o número de elementos de um conjunto com a função **len**
- ⌈ O método **isdisjoint**(B) é chamado a partir de um conjunto A e retorna **True** se $A \cap B = \emptyset$, ou seja, se a interseção dos dois conjuntos é vazia, e **False** caso contrário
- ⌈ O método **issubset**(B) é chamado a partir de um conjunto A e retorna **True** se todo elemento de A também é elemento de B e **False** caso contrário
- ⌈ O método **issuperset**(B) é chamado a partir de um conjunto A e retorna **True** se todo elemento de B também é elemento de A e **False** caso contrário

Exemplo de aplicação de conjuntos

- ⌋ Um tautograma é uma sentença em que todas as palavras começam com a mesma letra. Escreva uma função em Python que verifique se uma sentença é um tautograma ou não.

Exemplo de aplicação de conjuntos

- ⌋ Um tautograma é uma sentença em que todas as palavras começam com a mesma letra. Escreva uma função em Python que verifique se uma sentença é um tautograma ou não.

```
def tautogram (sentenca) :  
    if isinstance(sentenca, str) :  
        palavras = sentenca.split()  
        iniciais = set()  
        for palavra em palavras:  
            iniciais.add(palavra[0].lower())  
        if len(iniciais) == 1 : return True  
    return False
```

```
resposta = tautogram("Silva selou sua sina")  
print(resposta)
```

True

⌋ **Dicionário** é nome dado a um mapeamento chave/valor

- * A estrutura é indexada a partir de dados considerados *hashables*, ou seja, objetos que possuem um valor de hash fixo que nunca muda durante toda a sua existência
- * É uma coleção mutável, assim como as listas e conjuntos
- * Eles são altamente eficientes para buscas e ideais para representar objetos ou registros onde cada dado é associado a um identificador exclusivo

⌋ Objetos imutáveis podem ser utilizados como chaves de um dicionário

- * Em outras palavras, *strings*, números inteiros e reais, tuplas e booleanos

☾ Há quatro maneiras de criar um dicionário no Python

- ✱ **Dicionário vazio:** atribuímos a uma variável um par de chaves `{}` ou então o construtor **`dict()`**
- ✱ **Construção explícita:** podemos listar os elementos do dicionário como um par **chave:valor** entre as chaves ou então usando o construtor **`dict`** passando informações como argumentos nomeados
- ✱ **Via lista de tuplas:** possuímos uma lista de tuplas de tamanho 2 e criamos o dicionário a partir de um *casting* para a classe **`dict`**. O primeiro elemento do par se torna a chave e o segundo elemento o valor
- ✱ **Dictionary Comprehension**

☾○○☾ **IMPORTANTE:** Dicionários não admitem chaves repetidas

Dictionary comprehension

- Podemos definir um dicionário a partir de uma outra coleção usando o seguinte modelo

{**chave** : **valor** **for** **variável** **in** **coleção** **if** **condição**}

- * **chave** é a expressão que define o formato das chaves
- * **valor** é a expressão que define o formato dos valores
- * **variável** é a variável usada no laço **for**
- * **coleção** é a coleção usada como base
- * **condição** é uma condição que os valores de **variável** devem atender (parte opcional)

- Podemos nos valer de duplas para criar o dicionário usando a função **zip**

Exemplo de criação de dicionários

```
dic1 = {4:"lua", "sol":12, 40:55}

dic2 = dict(nome="Maurício",curso="Computação")
# {"nome" : "Maurício", "curso" : "Computação"}

lista = [("sol",18),("lua",17),("torre",16)]
dic3 = dict(lista)
# {"sol" : 18, "lua" : 17, "torre" : 16}

dic4 = {x:2*x for x in range(1,6)}
# {1: 2, 2: 4, 3: 6, 4: 8, 5: 10}

list1 = ["sol","lua","estrela","torre"]
list2 = [18,17,19,16]
dic5 = {x:y for x,y in zip(list1,list2)}
# {"sol": 18, "lua": 17, "estrela": 19, "torre": 16}
```

﴿ Podemos adicionar dados ao dicionário de duas maneiras

- ✱ **Uma adição por vez:** faça uma atribuição do tipo **dic[chave]=valor**
- ✱ **Adicionar vários dados novos:** a partir do dicionário, chame a função **update** passando como parâmetro o dicionário que possui os pares que você deseja adicionar

```
dic1 = {"lua":4, "flor":10, "cor":0}
dic2 = {"sol":19, "lua":33}
dic1["sal"] = 7
# {"lua":4, "flor":10, "cor":0, "sal":7}
dic1["lua"] = 13
# {"lua":13, "flor":10, "cor":0, "sal":7}
dic1.update(dic2)
# {"lua":33, "flor":10, "cor":0, "sal":7, "sol":19}
```

Alteração de dados

- ⌋ A alteração é muito semelhante à inserção de um dado
 - * Se a chave usada não existe, o dicionário a adiciona junto com o valor usado
 - * Se a chave usada existe, o valor dela é alterado para aquele que foi utilizado
- ⌋ Ao fazer um **update**, se um dicionário 1 compartilhar alguma chave com um dicionário 2, então o valor final dessa chave depende de qual deles é o argumento do **update**

```
dic1 = {"lua":4, "flor":10, "cor":0}
```

```
dic2 = {"sol":19, "lua":33}
```

```
dic1.update(dic2)
```

```
# {"lua":33, "flor":10, "cor":0, "sol":19}
```

☾ A remoção de dados pode ser feita de quatro maneiras

- * **pop(chave)** : método chamado a partir de um dicionário, retorna o valor associado à chave e remove o par chave/valor do dicionário (não gera erro se a chave não existir)
- * **popitem()** : método chamado a partir de um dicionário, retorna o par (chave, valor) inserido por último no dicionário e remove o par chave/valor do dicionário (gera erro se o dicionário estiver vazio)
- * **clear()** : método chamado a partir de um dicionário e esvazia o dicionário
- * **del dicionario[chave]** : apaga o par chave/valor dada a chave indicada (gera erro se a chave não existir)

Outras operações

- ⌈ Seja **dic** um dicionário. Outros métodos disponíveis para dicionários são
 - * **dic.get(ch)** : retorna o valor associado à chave *ch* (se a chave não existe, retorna **None**)
 - * **dic.copy()** : retorna uma cópia superficial do dicionário (*shallow copy*)
 - * **dic.keys()** : retorna uma coleção das chaves do dicionário
 - * **dic.values()** : retorna uma coleção dos valores do dicionário
 - * **dic.items()** : retorna uma coleção dos pares chave/valor do dicionário
- ⌈ Embora dicionários sejam iteráveis, tipicamente iremos acessar as chaves, os valores ou os pares chave/valor que existem nele

